

Name : Adhithya.S
Reg. No: 192412352

CODE:

```
import pandas as pd
import math

# Training Dataset
data = pd.DataFrame({
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
               'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
               'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                   'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
                   'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                'Normal', 'Normal', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes',
                  'No', 'Yes', 'No', 'Yes', 'Yes',
                  'Yes', 'Yes', 'Yes', 'No']
})

# Calculate Entropy
def entropy(target):

    values = target.value_counts()
    total = len(target)

    entropy_value = 0

    for count in values:
        probability = count / total
        entropy_value -= probability * math.log2(probability)

    return entropy_value

# Calculate Information Gain
def information_gain(data, attribute, target):

    total_entropy = entropy(data[target])

    values = data[attribute].unique()

    weighted_entropy = 0

    for value in values:

        subset = data[data[attribute] == value]

        weighted_entropy += (
            len(subset) / len(data)
```

```

    ) * entropy(subset[target])

return total_entropy - weighted_entropy

# Build ID3 Decision Tree
def id3(data, attributes, target):

    # If all samples belong to the same class
    if len(data[target].unique()) == 1:
        return data[target].iloc[0]

    # If no attributes are left
    if len(attributes) == 0:
        return data[target].mode()[0]

    # Find attribute with maximum Information Gain
    gains = {}

    for attribute in attributes:
        gains[attribute] = information_gain(
            data, attribute, target
        )

    best_attribute = max(gains, key=gains.get)

    tree = {best_attribute: {}}

    # Create branches
    for value in data[best_attribute].unique():

        subset = data[data[best_attribute] == value]

        remaining_attributes = [
            attr for attr in attributes
            if attr != best_attribute
        ]

        subtree = id3(
            subset,
            remaining_attributes,
            target
        )

        tree[best_attribute][value] = subtree

    return tree

# Classify a New Sample
def classify(tree, sample):

    if not isinstance(tree, dict):
        return tree

    attribute = list(tree.keys())[0]

    value = sample[attribute]

    if value not in tree[attribute]:
        return "Unknown"

    return classify(
        tree[attribute][value],

```

```

        sample
    )

# Attributes used for building the tree
attributes = [
    'Outlook',
    'Temperature',
    'Humidity',
    'Wind'
]

target = 'PlayTennis'

# Build the Decision Tree
tree = id3(data, attributes, target)

# Display Decision Tree
print("\nDecision Tree:")
print(tree)

# New sample to classify
new_sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

# Classify new sample
result = classify(tree, new_sample)

print("\nNew Sample:")
print(new_sample)

print("\nPredicted Class:", result)

```

OUTPUT :

```

Decision Tree:
{'Outlook': {'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}, 'Overcast': 'Yes', 'Rain': {'Wind': {'Weak': 'Yes', 'Strong': 'No'}}}}

New Sample:
{'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Predicted Class: No

```